

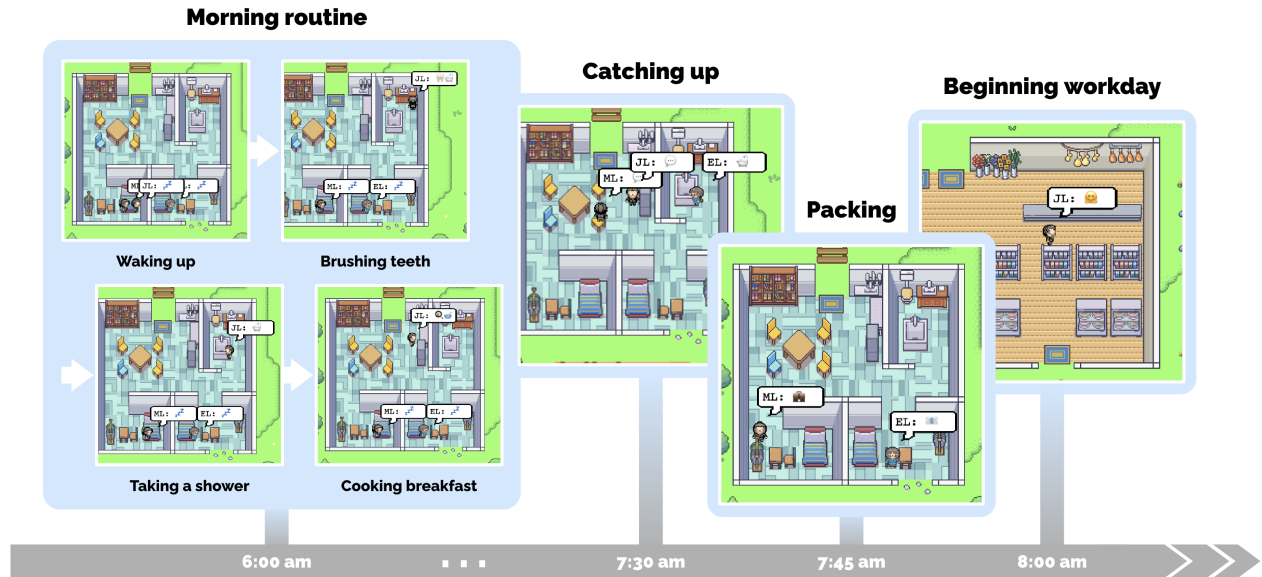


Figure 3: A morning in the life of a generative agent, John Lin. John wakes up around 6 am and completes his morning routine, which includes brushing his teeth, taking a shower, and eating breakfast. He briefly catches up with his wife, Mei, and son, Eddy, before heading out to begin his workday.



Figure 4: At the beginning of the simulation, one agent is initialized with an intent to organize a Valentine’s Day party. Despite many possible points of failure in the ensuing chain of events—agents might not act on that intent, might forget to tell others, might not remember to show up—the Valentine’s Day party does, in fact, occur, with a number of agents gathering and interacting.

Later that day, after Sam left, Tom and John, who heard the news from another source, discuss Sam’s chances of winning the election:

John: I heard that Sam Moore is running for mayor in the local election. Do you think he has a good chance of winning?

Tom: I do think he has a good chance. He’s been working hard in the community and I think he will get a lot of support. What do you think?

John: I think it’s great that he’s running. I’m curious to see how the election will turn out.

Gradually, Sam’s candidacy becomes the talk of the town, with some supporting him and others remaining undecided.

3.4.2 Relationship Memory. Agents in Smallville form new relationships over time and remember their interactions with other agents. For example, at the start, Sam does not know Latoya Williams. While taking a walk in Johnson Park, Sam runs into Latoya, and they introduce themselves. Latoya mentions that she is working on a photography project: “I’m here to take some photos for a project I’m working on.” In a later interaction, Sam’s interactions with Latoya indicate a memory of that interaction, as he asks “Hi, Latoya. How is your project going?” and she replies “Hi, Sam. It’s going well!”

3.4.3 Coordination. Generative agents coordinate with each other. Isabella Rodriguez, at Hobbs Cafe, is initialized with an intent to plan a Valentine’s Day party from 5 to 7 p.m. on February 14th. From this seed, the agent proceeds to invite friends and customers when she sees them at Hobbs Cafe or elsewhere. Isabella then spends the afternoon of the 13th decorating the cafe for the occasion. Maria, a frequent customer and close friend of Isabella’s, arrives at the cafe. Isabella asks for Maria’s help in decorating for the party, and Maria agrees. Maria’s character description mentions that she has a crush on Klaus. That night, Maria invites Klaus, her secret crush, to join her at the party, and he gladly accepts.

On Valentine’s Day, five agents, including Klaus and Maria, show up at Hobbs Cafe at 5 pm, and they enjoy the festivities (Figure 4). In this scenario, the end user only set Isabella’s initial intent to throw a party and Maria’s crush on Klaus: the social behaviors of spreading the word, decorating, asking each other out, arriving at the party, and interacting with each other at the party were initiated by the agent architecture.

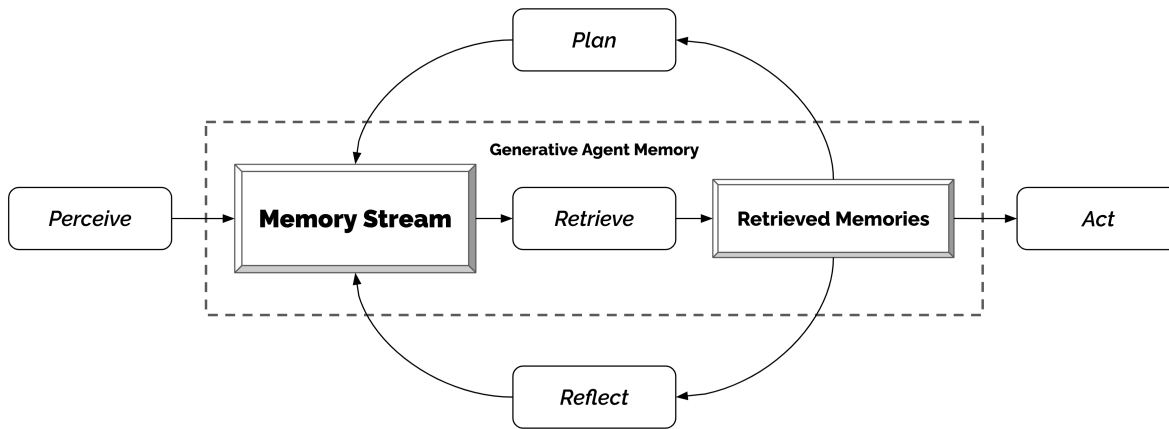


Figure 5: Our generative agent architecture. Agents perceive their environment, and all perceptions are saved in a comprehensive record of the agent’s experiences called the memory stream. Based on their perceptions, the architecture retrieves relevant memories and uses those retrieved actions to determine an action. These retrieved memories are also used to form longer-term plans and create higher-level reflections, both of which are entered into the memory stream for future use.

4 GENERATIVE AGENT ARCHITECTURE

Generative agents aim to provide a framework for behavior in an open world: one that can engage in interactions with other agents and react to changes in the environment. Generative agents take their current environment and past experiences as input and generate behavior as output. Underlying this behavior is a novel agent architecture that combines a large language model with mechanisms for synthesizing and retrieving relevant information to condition the language model’s output. Without these mechanisms, large language models can output behavior, but the resulting agents may not react based on the agent’s past experiences, may not make important inferences, and may not maintain long-term coherence. Challenges with long-term planning and coherence remain [19] even with today’s most performant models such as GPT-4. Because generative agents produce large streams of events and memories that must be retained, a core challenge of our architecture is to ensure that the most relevant pieces of the agent’s memory are retrieved and synthesized when needed.

At the center of our architecture is the memory stream, a database that maintains a comprehensive record of an agent’s experience. From the memory stream, records are retrieved as relevant to plan the agent’s actions and react appropriately to the environment. Records are recursively synthesized into higher- and higher-level reflections that guide behavior. Everything in the architecture is recorded and reasoned over as a natural language description, allowing the architecture to leverage a large language model.

Our current implementation utilizes the gpt3.5-turbo version of ChatGPT [77]. We expect that the architectural basics of generative agents—memory, planning, and reflection—will likely remain the same as language models improve. Newer language models (e.g., GPT-4) will continue to expand the expressive power and performance of the prompts that underpin generative agents. As of writing, however, GPT-4’s API was invitation-only, so our agents use ChatGPT.

4.1 Memory and Retrieval

Challenge: Creating generative agents that can simulate human behavior requires reasoning about a set of experiences that is far larger than what should be described in a prompt, as the full memory stream can distract the model and does not even currently fit into the limited context window. Consider the Isabella agent answering the question, “What are you passionate about these days?” Summarizing all of Isabella’s experiences to fit in the limited context window of the language model produces an uninformative response, where Isabella discusses topics such as collaborations for events and projects and cleanliness and organization in a cafe. Instead of summarizing, the memory stream described below surfaces relevant memories, resulting in a more informative and specific response that mentions Isabella’s passion for making people feel welcome and included, planning events and creating an atmosphere that people can enjoy, such as the Valentine’s Day party.

Approach: The *memory stream* maintains a comprehensive record of the agent’s experience. It is a list of memory objects, where each object contains a natural language description, a creation timestamp, and a most recent access timestamp. The most basic element of the memory stream is an *observation*, which is an event directly perceived by an agent. Common observations include behaviors performed by the agent themselves or behaviors that agents perceive being performed by other agents or non-agent objects. For instance, Isabella Rodriguez, who works at a coffee shop, might accrue the following observations over time: (1) Isabella Rodriguez is setting out the pastries, (2) Maria Lopez is studying for a Chemistry test while drinking coffee, (3) Isabella Rodriguez and Maria Lopez are conversing about planning a Valentine’s day party at Hobbs Cafe, (4) The refrigerator is empty.

Our architecture implements a retrieval function that takes the agent’s current situation as input and returns a subset of the memory stream to pass on to the language model. There are many possible implementations of a retrieval function, depending on what is important for the agent to consider when deciding how to act.

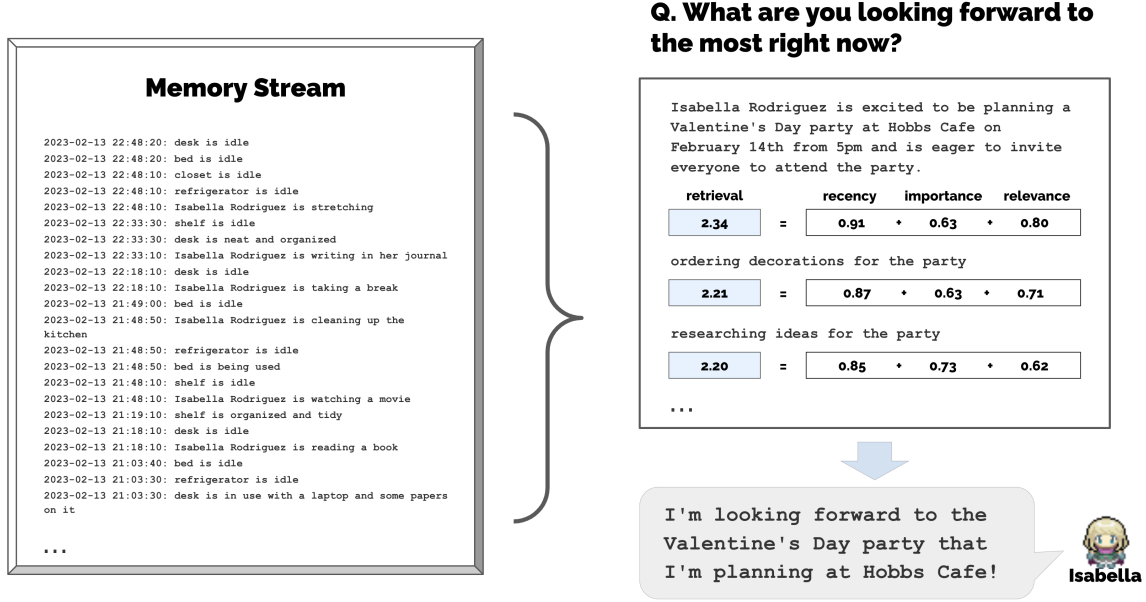


Figure 6: The memory stream comprises a large number of observations that are relevant and irrelevant to the agent’s current situation. Retrieval identifies a subset of these observations that should be passed to the language model to condition its response to the situation.

In our context, we focus on three main components that, together, produce effective results.

Recency assigns a higher score to memory objects that were recently accessed, so that events from a moment ago or this morning are likely to remain in the agent’s attentional sphere. In our implementation, we treat recency as an exponential decay function over the number of sandbox game hours since the memory was last retrieved. Our decay factor is 0.995.

Importance distinguishes mundane from core memories by assigning a higher score to memory objects that the agent believes to be important. For instance, a mundane event, such as eating breakfast in one’s room, would yield a low importance score, whereas a breakup with one’s significant other would yield a high score. There are many possible implementations of an importance score; we find that directly asking the language model to output an integer score is effective. The full prompt appears below:

On the scale of 1 to 10, where 1 is purely mundane (e.g., brushing teeth, making bed) and 10 is extremely poignant (e.g., a break up, college acceptance), rate the likely poignancy of the following piece of memory.

Memory: buying groceries at The Willows Market and Pharmacy

Rating: <fill in>

This prompt returns an integer value of 2 for “cleaning up the room” and 8 for “asking your crush out on a date.” The importance score is generated at the time the memory object is created.

Relevance assigns a higher score to memory objects that are related to the current situation. What is relevant depends on the answer to, “Relevant to what?”, so we condition relevance on a

query memory. If the query, for example, is that a student is discussing what to study for a chemistry test with a classmate, memory objects about their breakfast should have low relevance, whereas memory objects about the teacher and schoolwork should have high relevance. In our implementation, we use the language model to generate an embedding vector of the text description of each memory. Then, we calculate relevance as the cosine similarity between the memory’s embedding vector and the query memory’s embedding vector.

To calculate the final retrieval score, we normalize the recency, relevance, and importance scores to the range of [0, 1] using min-max scaling. The retrieval function scores all memories as a weighted combination of the three elements: $score = \alpha_{recency} \cdot recency + \alpha_{importance} \cdot importance + \alpha_{relevance} \cdot relevance$. In our implementation, all α s are set to 1. The top-ranked memories that fit within the language model’s context window are included in the prompt.

4.2 Reflection

Challenge: Generative agents, when equipped with only raw observational memory, struggle to generalize or make inferences. Consider a scenario in which Klaus Mueller is asked by the user: “If you had to choose one person of those you know to spend an hour with, who would it be?” With access to only observational memory, the agent simply chooses the person with whom Klaus has had the most frequent interactions: Wolfgang, his college dorm neighbor. Unfortunately, Wolfgang and Klaus only ever see each other in passing, and do not have deep interactions. A more desirable response requires that the agent generalize from memories of Klaus spending hours on a research project to generate a higher-level reflection that Klaus is passionate about research, and likewise